

# CS336 作业 5 (alignment) : 对齐与推理强化学习

Version 1.0.2 · CS336 Staff · 2025 年春季

## 1 作业概览

在本作业中，你将获得一些实践经验：训练语言模型在求解数学问题时进行推理。

你将实现的内容。

1. 针对竞赛数学问题 MATH 数据集 Hendrycks et al. [2021] 的零样本提示基线。
2. 使用更强推理模型 DeepSeek R1 (DeepSeek-AI et al. 2025) 生成的推理轨迹进行监督微调。
3. 使用可验证奖励改进推理性能的专家迭代 (Expert Iteration)。
4. 使用可验证奖励改进推理性能的 Group-Relative Policy Optimization (GRPO)。

对于感兴趣的同学，我们还会发布一个完全可选的部分，主题是将语言模型与人类偏好对齐；该部分将在未来几天发布。

你将运行的实验。

1. 测量 Qwen 2.5 Math 1.5B 的零样本提示性能（我们的基线）。
2. 使用来自 R1 的推理轨迹在 Qwen 2.5 Math 1.5B 上运行 SFT。
3. 使用可验证奖励在 Qwen 2.5 Math 1.5B 上运行专家迭代。
4. 使用可验证奖励在 Qwen 2.5 Math 1.5B 上运行 GRPO。

代码结构。所有作业代码以及本文档都可以在 GitHub 上获得：

[github.com/stanford-cs336/assignment5-alignment](https://github.com/stanford-cs336/assignment5-alignment)

请 git clone 该仓库。如果有任何更新，我们会通知你，你可以通过 git pull 获取最新版本。

1. `cs336_alignment/*`：这是你编写作业 5 代码的位置。注意，这里除了少量 starter code 之外没有现成代码，因此你可以从头实现任何你想要的结构。
2. `cs336_alignment/prompts/*`：为方便起见，我们提供了包含提示词的文本文件，以尽量减少从 PDF 复制粘贴提示词到代码时可能产生的错误。
3. `tests/*.py`：这里包含你必须通过的所有测试。你只需要通过 `tests/test_sft.py` 和 `tests/test_grpo.py` 中的测试；其余测试是非必做部分的测试。这些测试会调用 `tests/adapters.py` 中定义的 hooks。你需要实现 adapters，把你的代码连接到测试。为了调试，编写更多测试或修改测试代码可能会有帮助，但你的实现最终应当通过原始提供的测试套件。
4. `README.md`：该文件包含一些设置环境的基本说明。

你可以使用的工具。我们希望你从头构建大部分与 RL 相关的组件。你可以使用 vLLM

之类的工具从语言模型生成文本 (§ 3.1)。此外，你可以使用 HuggingFace Transformers 加载 Qwen 2.5 Math 1.5B 模型和 tokenizer，并运行前向传播 (§ 4.1)，但不能使用任何训练工具（例如 `Trainer` 类）。

提交方式。你需要向 Gradescope 提交以下文件：

- `writeup.pdf`：回答所有书面问题。请用排版后的形式提交你的答案。

- `code.zip`：包含你编写的所有代码。

## 2 使用语言模型进行推理

### 2.1 动机

语言模型的一个显著用例，是构建能够处理广泛自然语言处理任务的通用系统。在本作业中，我们将关注语言模型的一个发展中用例：数学推理。它将作为测试平台，让我们设置评测、进行监督微调，并实验如何用强化学习（RL）教会语言模型推理。

与之前的作业相比，这次会有两点不同。

- 第一，我们不会使用之前作业中的语言模型代码库和模型。理想情况下，我们希望使用之前作业中训练的基础语言模型，但微调这些模型不会得到令人满意的结果——这些模型太弱，无法展现出非平凡的数学推理能力。因此，我们将切换到一个现代的高性能语言模型（Qwen 2.5 Math 1.5B Base），并在该模型之上完成大部分工作。
- 第二，我们将引入一个新的基准，用于评估我们的语言模型。到目前为止，我们一直接受这样的观点：交叉熵是许多下游任务的良好替代指标。然而，本作业的目标是弥合基础模型与下游任务之间的差距，因此我们必须使用独立于交叉熵的评测。我们将使用 Hendrycks et al. [2021] 的 MATH 12K 数据集，其中包含具有挑战性的高中竞赛数学题。我们会通过将语言模型输出与参考答案进行比较来评估它。

### 2.2 思维链推理与推理 RL

语言模型中的一个令人兴奋的近期趋势，是使用思维链推理（chain-of-thought reasoning）来提升各种任务的性能。思维链指的是逐步推理一个问题的过程：在给出最终答案之前，先生成中间推理步骤。

LLM 的思维链推理。早期的思维链方法通过使用“scratchpad”将问题拆分为中间步骤，来微调语言模型求解简单数学任务（如算术）[Nye et al., 2021]。其他工作会提示强模型在回答前“think step by step”，并发现这显著提升了模型在数学推理任务（如小学数学题）上的表现 [Wei et al., 2023]。

通过专家迭代学习推理。Self-Taught Reasoner (STaR) [Zelikman et al., 2022] 将推理表述为一个自举循环：预训练模型首先采样多样的思维链（CoT），只保留那些导向正确答案的轨迹，然后在这些“专家”轨迹上微调。反复迭代该循环可以提升语言模型的推理能力和解题率。STaR 展示了这种专家迭代 [Anthony et al., 2017] 的版本：通过自动的、基于字符串匹配的生成答案验证，可以在没有人工书写推理轨迹的情况下自举推理技能。

使用可验证奖励的推理 RL、o1 与 R1。

近期工作探索了使用更强大的强化学习算法和可验证奖励来提升推理性能。OpenAI 的 o1（以及后续 o3/o4）[OpenAI et al., 2024]、DeepSeek 的 R1 [DeepSeek-AI et al., 2025] 和 Moonshot 的 kimi k1.5 [Team et al., 2025] 使用策略梯度方法 [Sutton et al., 1999]，在可以通过字符串匹配或单元测试验证正确性的数学与代码任务上训练，展示了在竞赛数学和编程性能上的显著提升。后续工作如 Open-R1 [Face, 2025]、SimpleRL-Zoo [Zeng et al., 2025] 和 TinyZero [Pan et al., 2025] 也确认，即使对小到 1.5B 参数的模型，使用可验证奖励的纯强化学习也能提升推理性能。

我们的设置：模型与数据集。在接下来的章节中，我们将考虑越来越复杂的方法，训练一个基础语言模型逐步推理，从而解决数学问题。在本作业中，我们将使用 Qwen 2.5 Math 1.5B Base 模型。该模型是在 Qwen 2.5 1.5B 模型基础上，使用高质量合成数学预训练数据继续预训练得到的 [Yang et al., 2024]。MATH 数据集可在 Together 集群的 `/data/a5-alignment/MATH` 获取。

给开源审阅者的提示：替代数据集

遗憾的是，由于版权声明，MATH

数据集目前并不公开。如果你是在家跟做，可以使用以下开源数学推理数据集之一：

- Countdown [Pan et al., 2025]，可在此处获得：一个基于英国电视节目 Countdown 的简单合成任务，常被用作小规模推理 RL 的测试平台。
- GSM8K [Cobbe et al., 2021a]，可在此处获得：小学数学文字题，比 MATH 更简单，但应足以帮助你调试正确性并熟悉推理 RL 流水线。
- Tulu 3 SFT Math [Lambert et al., 2025]，可在此处获得：使用 GPT-4o 和 Claude 3.5 Sonnet 生成的合成数学题。由于它们是合成的，一些答案（甚至问题本身）可能并不完全正确。
- 此处链接的其他数学 SFT 数据集。

如果数据集没有直接提供短形式的 ground-truth 标签（例如  $1/2$ ），你可以使用 Math-Verify 等数学答案解析器处理 ground-truth 列来获得它们。

## 3 测量零样本 MATH 性能

我们将首先测量基础语言模型在 MATH 的 5K

样例测试集上的性能。建立这一基线有助于理解后续每种方法如何影响模型行为。

除非另有说明，对于 MATH 上的实验，我们将使用来自 DeepSeek R1-Zero 模型 [DeepSeek-AI et al., 2025] 的如下提示词。我们将其称为 **r1\_zero** 提示词：

```
A conversation between User and Assistant. The User asks a question, and the Assistant solves it. The Assistant first thinks about the reasoning process in the mind and then provides the User with the answer. The reasoning process is enclosed within <think> </think> and answer is enclosed within <answer> </answer> tags, respectively, i.e., <think> reasoning process here </think> <answer> answer here </answer>.
User: {question}
Assistant: <think>
```

**r1\_zero** 提示词位于文本文件 `cs336_alignment/prompts/r1_zero.prompt` 中。

在提示词中，**question** 指的是我们插入的某个问题，例如：“Natalia sold clips to 48 of her friends in April, and then she sold half as many clips in May. How many clips did Natalia sell altogether in April and May? ”。期望是模型扮演 assistant 的角色，并开始生成思考过程（因为我们已经包含了左侧的 think 标签 `<think>`），然后用 `</think>` 关闭思考过程，最后在 `answer` 标签中生成一个最终的符号答案，例如 `<answer> 4x + 10 </answer>`。让模型生成 `<answer> </answer>` 这样的标签的目的，是便于我们解析模型输出并将其与 ground truth 答案比较，同时也可以看到在正确的答案结束标签 `</answer>` 时停止生成。

关于提示词选择的说明。事实证明，**r1\_zero** 提示词并不是在 RL 后最大化下游性能的最佳选择，因为它与 Qwen 2.5 Math 1.5B 的预训练方式存在不匹配。Liu et al. [2025]

发现，仅用问题本身（不加其他内容）提示模型，一开始就能达到很高的准确率，例如能匹配 **r1\_zero** 提示词经过 100 多步 RL 之后的表现。他们的发现表明，Qwen 2.5 Math 1.5B 已经在此类问答对上预训练过。不过，我们在本作业中选择 **r1\_zero** 提示词，是因为使用它进行 RL 时，在很少步数内就能看到清晰的准确率提升，这让我们能够快速走过 RL 的机制并 sanity check 正确性，即使我们没有得到最优的最终性能。作为现实检查，你将在作业后面对 **question\_only** 提示词进行直接比较。

### 3.1 使用 vLLM 进行离线语言模型推理

为了评估语言模型，我们需要为各种提示词生成续写（responses）。当然，你可以自己实现生成函数（例如你在作业 1 中所做的那样），但高效实现 RL

需要高性能推理技术，而实现这些推理技术超出了本作业范围。因此，在本作业中，我们建议使用 vLLM 进行离线批量推理。vLLM 是一个高吞吐、内存高效的语言模型推理引擎，集成了多种有用的效率技术（例如优化 CUDA kernels、用于高效 attention KV cache 的 PagedAttention [Kwon et al., 2023] 等）。要使用 vLLM 为一组提示词生成续写：

```
from vllm import LLM, SamplingParams

# Sample prompts.
prompts = [
    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]

# Create a sampling params object, stopping generation on newline.
sampling_params = SamplingParams(
    temperature=1.0, top_p=1.0, max_tokens=1024, stop=["\n"]
)

# Create an LLM.
llm = LLM(model=<path to model>)

# Generate texts from the prompts. The output is a list of RequestOutput objects
# that contain the prompt, generated text, and other information.
outputs = llm.generate(prompts, sampling_params)

# Print the outputs.
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

在上面的示例中，LLM 可以用 HuggingFace 模型名称初始化（如果本地找不到，会自动下载并缓存），也可以用 HuggingFace 模型路径初始化。由于下载可能很慢（尤其是更大的模型，例如 70B 参数），并且为了节省集群磁盘空间（避免每个人都独立保存一份预训练模型），我们已经在 Together 集群的以下路径下载好了这些预训练模型。请不要在 Together 集群上重新下载这些模型：

- Qwen 2.5 Math 1.5B Base（用于推理实验）：`/data/a5-alignment/models/Qwen2.5-Math-1.5B`
- Llama 3.1 8B Base（用于可选的指令微调实验）：`/data/a5-alignment/models/Llama-3.1-8B`
- Llama 3.3 70B Instruct（用于可选的指令微调实验）：`/data/a5-alignment/models/Llama-3.3-70B-Instruct`

## 3.2 零样本 MATH 基线

提示设置。为了评估 MATH 测试集上的零样本性能，我们将简单加载样例，并使用上文的 `r1_zero` 提示词提示语言模型回答问题。

评估指标。

当我们评估一个多选或二分类响应任务时，评估指标很明确——我们测试模型是否输出了完全正确的答案。

在数学问题中，我们假设存在已知 ground truth（例如 `0.5`），但我们不能简单地测试模型是否精确输出 `0.5`——它也可能回答 `<answer> 1/2 </answer>`。因此，在评估 MATH 时，我们必须处理一个棘手问题：如何匹配语义等价的 LM 响应。

为此，我们希望设计一个答案解析函数，它以模型输出和已知 ground-truth 为输入，并返回一个布尔值，表示模型输出是否正确。例如，奖励函数可以接收一个以 `<answer> She sold 15 clips. </answer>` 结尾的模型字符串输出，以及 gold answer `72`，然后在模型输出正确时返回

True, 否则返回 False (在这个例子中应返回 False)。

对于 MATH 实验, 我们将使用近期推理 RL 工作 [Liu et al., 2025] 中使用的一个快速且相当准确的答案解析器。该奖励函数实现于 `cs336_alignment.drgrpo_grader.r1_zero_reward_fn`, 除非另有说明, 你应当使用它来评估 MATH 性能。

生成超参数。生成响应时, 我们将使用 temperature 1.0、top-p 1.0、最大生成长度 1024。提示词要求模型以字符串 `</answer>` 结束答案, 因此我们可以指示 vLLM 在模型输出该字符串时停止:

```
# Based on Dr. GRPO: stop when the model completes its answer
# https://github.com/sail-sg/understand-r1-zero/blob/
# c18804602b85da9e88b4aeeb6c43e2f08c594fbc/train_zero_math.py#L167
sampling_params.stop = ["</answer>"]
sampling_params.include_stop_str_in_output = True
```

#### 问题 (math\_baseline) : 4 分

(a) 编写一个脚本来评估 Qwen 2.5 Math 1.5B 在 MATH 上的零样本性能。该脚本应当 (1) 从 `/data/a5-alignment/MATH/validation.jsonl` 加载 MATH validation 样例, (2) 使用 `r1_zero` 提示词将它们格式化为语言模型的字符串 prompts, 并且 (3) 为每个样例生成输出。该脚本还应当 (4) 计算评估指标, 并且 (5) 将样例、模型生成结果以及对应的评估分数序列化到磁盘, 供后续问题分析使用。

你的实现中包含一个类似如下参数的 `evaluate_vllm` 方法可能会很有帮助, 因为稍后可以复用它:

```
def evaluate_vllm(
    vllm_model: LLM,
    reward_fn: Callable[[str, str], dict[str, float]],
    prompts: List[str],
    eval_sampling_params: SamplingParams
) -> None:
    """
    Evaluate a language model on a list of prompts,
    compute evaluation metrics, and serialize results to disk.
    """
```

交付物: 一个评估 baseline zero-shot MATH performance 的脚本。

(b) 在 Qwen 2.5 Math 1.5B 上运行你的评估脚本。以下每类中分别有多少模型生成结果: (1) format 和 answer reward 都为 1 的正确结果; (2) format reward 为 1、answer reward 为 0; (3) format reward 为 0、answer reward 为 0? 观察至少 10 个 format reward 为 0 的案例, 你认为问题出在基础模型输出上, 还是解析器上? 为什么? 对于至少 10 个 format reward 为 1 但 answer reward 为 0 的案例, 又如何?

交付物: 对模型和奖励函数表现的评论, 包括每个类别的示例。

(c) Qwen 2.5 Math 1.5B 的零样本基线在 MATH 上表现如何?

交付物: 1-2 句评估指标说明。

## 4 MATH 的监督微调

算法 1 监督微调 (SFT)

输入: 初始策略模型  $\pi_{\theta init}$ ; SFT 数据集  $D$

1. 策略模型  $\pi_{\theta} \leftarrow \pi_{\theta init}$
2. 对  $step = 1, \dots, n_{sft\_steps}$ :

3. 从  $\mathbf{D}$  中采样一批 question-response pairs  $\mathbf{D}_b$
4. 使用模型  $\pi_\theta$  计算 responses 在 questions 条件下的交叉熵损失
5. 对交叉熵损失执行一次梯度步，更新模型参数  $\theta$
6. 结束循环

输出： $\pi_\theta$

用于推理的监督微调。在本节中，我们将在 MATH 数据集上微调基础模型（算法 1）。由于我们的目标是提升模型的推理能力，而不是微调模型直接预测正确答案，因此我们会微调它先生成思维链推理轨迹，然后生成答案。为此，我们提供了这样一个推理轨迹数据集，轨迹来自 DeepSeek R1 DeepSeek-AI et al. [2025]：

`/data/a5-alignment/MATH/sft.jsonl`

在实际训练推理模型时，SFT 通常被用作第二阶段 RL 微调的 warm-start。主要有两个原因。第一，SFT 需要高质量标注数据（即已有推理轨迹），而 RL 只需要正确答案作为反馈。第二，即便在标注数据充足的场景中，RL 仍可能通过找到优于 SFT 数据的策略来解锁性能提升。遗憾的是，我们使用的模型不够大，无法在组合 SFT 和 RL 时显示出明显效果，因此本作业将这两个阶段分开处理。

## 4.1 使用 HuggingFace 模型

加载 HuggingFace 模型和 tokenizer。要从本地目录加载 HuggingFace 模型和 tokenizer（使用 bfloat16 与 FlashAttention-2 以节省内存），可以使用以下 starter code：

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "/data/a5-alignment/models/Qwen2.5-Math-1.5B",
    torch_dtype=torch.bfloat16,
    attn_implementation="flash_attention_2",
)
tokenizer = AutoTokenizer.from_pretrained("/data/a5-alignment/models/Qwen2.5-Math-1.5B")
```

前向传播。加载模型后，我们可以在一批 input IDs 上运行前向传播，并通过输出的 `.logits` 属性获得 logits。然后，我们可以计算模型预测 logits 与实际 labels 之间的 loss：

```
input_ids = train_batch["input_ids"].to(device)
labels = train_batch["labels"].to(device)
logits = model(input_ids).logits
loss = F.cross_entropy(..., ...)
```

保存训练后的模型。训练结束后，要将模型保存到某个目录，可以使用 `.save_pretrained()` 函数，并传入目标输出目录路径。请确保保存在 `/data/yourusername` 下，因为模型可能很大。我们建议同时保存 tokenizer（即使你没有修改它），这样模型和 tokenizer 都能从同一个目录自包含地加载。

```
# Save the model weights
model.save_pretrained(save_directory=output_dir)
tokenizer.save_pretrained(save_directory=output_dir)
```

梯度累积。尽管我们使用 bfloat16 加载模型并使用 FlashAttention-2，即使 80GB GPU 也没有足够内存支持合理的 batch size。为了使用更大的 batch size，我们可以使用一种称为梯度累积的技术。基本思想是：不在每个 batch 后立即更新模型权重（即执行 optimizer step），而是先在多个 batch 上累积梯度，再执行一次梯度步。直观地说，如果我们有更大的 GPU，那么一次性在 32 个样例的 batch 上计算梯度，应该与将其拆成 16 个每批 2 个样例的小 batch、最后取平均得到相同结果。

在 PyTorch 中实现梯度累积很直接。回忆一下，每个权重张量都有一个 `.grad` 属性用于存储梯度。在调用 `loss.backward()` 之前，`.grad` 属性为 `None`。调用 `loss.backward()` 之后，`.grad` 属性包含梯度。通常，我们会执行 `optimizer.step()`，然后用 `optimizer.zero_grad()` 清零梯度，这会重置权重张量的 `.grad` 字段：

```
for inputs, labels in data_loader:
    # Forward pass.
    logits = model(inputs)
    loss = loss_fn(logits, labels)

    # Backward pass.
    loss.backward()

    # Update weights.
    optimizer.step()

    # Zero gradients in preparation for next iteration.
    optimizer.zero_grad()
```

为了实现梯度累积，我们只需要每 `k` 步调用一次 `optimizer.step()` 和 `optimizer.zero_grad()`，其中 `k` 是梯度累积步数。在调用 `loss.backward()` 前，我们将 `loss` 除以 `gradient_accumulation_steps`，从而让梯度在梯度累积步中取平均。

```
gradient_accumulation_steps = 4
for idx, (inputs, labels) in enumerate(data_loader):
    # Forward pass.
    logits = model(inputs)
    loss = loss_fn(logits, labels) / gradient_accumulation_steps

    # Backward pass.
    loss.backward()

    if (idx + 1) % gradient_accumulation_steps == 0:
        # Update weights every `gradient_accumulation_steps` batches.
        optimizer.step()

        # Zero gradients every `gradient_accumulation_steps` batches.
        optimizer.zero_grad()
```

因此，训练时的有效 batch size 会乘以 `k`，即梯度累积步数。

## 4.2 SFT 辅助方法

接下来，我们将实现一些辅助方法，在 SFT 以及后续 RL 实验中都会使用。

关于命名的简要说明：在接下来的章节中，我们会交替使用“output”“completion”和“response”来指模型在给定 prompt 下的补全。

Tokenizing prompts and outputs。对每个 question 和 target output 对 (`q`, `o`)，我们会分别 tokenize question 和 output，然后将它们拼接。接着，我们可以用 SFT 模型（或后续 RL policy）给 output 的 log-probabilities 打分。此外，我们需要构造一个 `response_mask`：一个布尔 mask，对 response 中的所有 token 为 `True`，对 question 和 padding token 为 `False`。我们将在训练循环中使用该 mask，确保只在 response token 上计算 loss。

**问题** (tokenize\_prompt\_and\_output)：Prompt 与 output tokenization（2 分）

交付物：实现一个 `tokenize_prompt_and_output` 方法，它分别 tokenize question 和 output 字符串，将它们拼接在一起，并构造 `response_mask`。推荐接口如下：

```
def tokenize_prompt_and_output(prompt_strs, output_strs, tokenizer):
```

Tokenize prompt 和 output 字符串，并构造一个 mask：response token 为 1，其他 token（prompt 或 padding）为 0。

参数：

- `prompt_strs: list[str]`: prompt 字符串列表。
- `output_strs: list[str]`: output 字符串列表。
- `tokenizer: PreTrainedTokenizer`: 用于 tokenization 的 tokenizer。

返回：

`dict[str, torch.Tensor]`。令 `prompt_and_output_lens` 为一个列表，包含 tokenized prompt 和 output 字符串的长度。那么返回字典应具有以下键：

- `input_ids`: 形状为  $(batch\_size, \max(prompt\_and\_output\_lens) - 1)$  的 `torch.Tensor`；由 prompt 和 output 字符串 tokenize 后拼接，并切掉最后一个 token。
- `labels`: 形状为  $(batch\_size, \max(prompt\_and\_output\_lens) - 1)$  的 `torch.Tensor`；shifted input ids，即去掉第一个 token 的 input ids。
- `response_mask`: 形状为  $(batch\_size, \max(prompt\_and\_output\_lens) - 1)$  的 `torch.Tensor`；labels 中 response token 的 mask。

为了测试你的代码，实现 `[adapters.run_tokenize_prompt_and_output]`。然后运行 `uv run pytest -k test_tokenize_prompt_and_output`，确保实现通过测试。

记录 per-token entropies。做 RL 时，跟踪 per-token entropies 通常很有用，可以观察模型的预测分布是否变得（过度）自信。我们现在实现它，并比较每种微调方法如何影响模型的预测熵。

离散分布  $p(x)$  在支持集  $X$  上的熵定义为：

$$H(p) = - \sum_{x \in X} p(x) \log p(x). \quad (1)$$

给定 SFT 或 RL 模型的 logits，我们将计算 per-token entropy，即每个 next-token prediction 的熵。

**问题** (`compute_entropy`)：Per-token entropy（1 分）

交付物：实现一个 `compute_entropy` 方法，计算 next-token predictions 的 per-token entropy。

推荐接口如下：

```
def compute_entropy(logits: torch.Tensor) -> torch.Tensor:
```

获取 next-token predictions 的 entropy（即在 vocabulary 维度上的 entropy）。

参数：

- `logits: torch.Tensor`: 形状为  $(batch\_size, sequence\_length, vocab\_size)$  的张量，包含未归一化 logits。

返回：

- `torch.Tensor`: 形状为  $(batch\_size, sequence\_length)$ 。每个 next-token prediction 的 entropy。

注意：你应当使用数值稳定的方法（例如使用 `logsumexp`）以避免 overflow。

为了测试你的代码，实现 `[adapters.run_compute_entropy]`。然后运行 `uv run pytest -k test_compute_entropy` 并确保测试通过。

从模型获取 log-probabilities。从模型获得 log-probabilities 是 SFT 和 RL 中都需要的 basic 操作。

对于 prefix  $x$ ，一个 LM 产生 next-token logits  $f_{\theta}(x) \in \mathbb{R}^{|V|}$ ，以及 label  $y \in V$ ， $y$  的 log-probability 为：

$$\log p_{\theta}(y \mid x) = \log [\text{softmax}(f_{\theta}(x))]_y \quad (2)$$

其中记号  $[x]_y$  表示向量  $x$  的第  $y$  个元素。

你会希望用数值稳定的方法来计算它，并且可以自由使用 `torch.nn.functional` 中的方法。我们也建议加入一个参数，用于可选地计算并返回 token entropies。

问题 (`get_response_log_probs`) : Response log-probs (以及 entropy) (2 分)

交付物：实现 `get_response_log_probs`，它从 causal language model 中获得 per-token conditional log-probabilities (给定之前 token)，并可选地获得模型 next-token distribution 的 entropy。

推荐接口如下：

```
def get_response_log_probs(
    model: PreTrainedModel,
    input_ids: torch.Tensor,
    labels: torch.Tensor,
    return_token_entropy: bool = False,
) -> dict[str, torch.Tensor]:
```

参数：

- `model: PreTrainedModel`：用于打分的 HuggingFace 模型（已放置到正确设备；如果不需要计算梯度，应处于 inference mode）。
- `input_ids: torch.Tensor`：形状为  $(batch\_size, sequence\_length)$ ，由 tokenization 方法产生的 prompt + response 拼接 token。
- `labels: torch.Tensor`：形状为  $(batch\_size, sequence\_length)$ ，由 tokenization 方法产生的 labels。
- `return_token_entropy: bool`：如果为 True，也通过调用 `compute_entropy` 返回 per-token entropy。

返回：

`dict[str, torch.Tensor]`。

- `"log_probs"`：形状为  $(batch\_size, sequence\_length)$ ，条件 log-probabilities  $\log p_\theta(x_t | x_{<t})$ 。
- `"token_entropy"`：可选，形状为  $(batch\_size, sequence\_length)$ ，每个位置的 per-token entropy（仅当 `return_token_entropy=True` 时存在）。

实现提示：

- 使用 `model(input_ids).logits` 获得 logits。

为了测试你的代码，实现 `[adapters.run_get_response_log_probs]`。然后运行 `uv run pytest -k test_get_response_log_probs` 并确保测试通过。

SFT microbatch train step。SFT 中最小化的 loss 是 target output 在 prompt 条件下的 negative log-likelihood。为计算此 loss，我们需要计算 target output 在 prompt 条件下的 log-probabilities，并对 output 中所有 token 求和，同时 mask 掉 prompt token 和 padding token。

我们将实现一个辅助函数，在后续 RL 中也会使用它。

**问题** (`masked_normalize`) : Masked normalize (1 分)

交付物：实现 `masked_normalize` 方法，它在尊重布尔 mask 的情况下，对 tensor 元素求和并用一个常数归一化。

推荐接口如下：

```
def masked_normalize(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    normalize_constant: float,
    dim: int | None = None,
) -> torch.Tensor:
```

沿某个维度求和并用常数归一化，只考虑 `mask == 1` 的元素。

参数：

- `tensor: torch.Tensor`：要求和并归一化的 tensor。
- `mask: torch.Tensor`：与 `tensor` 形状相同；值为 1 的位置会包含在求和中。
- `normalize_constant: float`：用于除法归一化的常数。
- `dim: int | None`：在归一化前求和的维度。如果为 `None`，则在所有维度上求和。

返回：

- `torch.Tensor`：归一化后的和；masked elements (`mask == 0`) 不参与求和。

为了测试你的代码，实现 `[adapters.run_masked_normalize]`。然后运行 `uv run pytest -k test_masked_normalize` 并确保通过。

SFT microbatch train step。现在我们已经准备好实现 SFT 的单个 micro-batch update（回忆一下，如果 `gradient_accumulation_steps > 1`，对于一个 train minibatch，我们会迭代多个 microbatches）。

**问题** (`sft_microbatch_train_step`)：Microbatch train step（3 分）

交付物：实现一个 SFT 的单个 micro-batch update，包括 cross-entropy loss、使用 `mask` 求和，以及 gradient scaling。

推荐接口如下：

```
def sft_microbatch_train_step(
    policy_log_probs: torch.Tensor,
    response_mask: torch.Tensor,
    gradient_accumulation_steps: int,
    normalize_constant: float = 1.0,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

在一个 microbatch 上执行 forward-and-backward pass。

参数：

- `policy_log_probs (batch_size, sequence_length)`：来自正在训练的 SFT policy 的 per-token log-probabilities。
- `response_mask (batch_size, sequence_length)`：response token 为 1，prompt/padding 为 0。
- `gradient_accumulation_steps`：每个 optimizer step 的 microbatches 数。
- `normalize_constant`：用于除以 sum 的常数。保持为 1.0 也可以。

返回：

`tuple[torch.Tensor, dict[str, torch.Tensor]]`。

- `loss`：标量 tensor。microbatch loss，已针对 gradient accumulation 调整。返回它是为了日志记录。
- `metadata`：来自底层 loss 调用的元数据，以及你希望记录的其他统计量。

实现提示：

- 你应当在该函数中调用 `loss.backward()`。确保对 gradient accumulation 做相应调整。

为了测试你的代码，实现 `[adapters.run_sft_microbatch_train_step]`。然后运行 `uv run pytest -k test_sft_microbatch_train_step` 并确认通过。

训练过程中记录生成结果。在训练过程中记录一些模型生成结果始终是好习惯，推理 SFT/RL 也不例外。编写一个 `log_generations` 函数，用于提示模型为给定 prompts（例如从 validation set 采样）生成 responses。建议至少为每个样例记录以下内容：

1. 输入 prompt。
2. SFT/RL 模型生成的 response。
3. ground-truth answer。
4. 奖励信息，包括 format、answer 和 total reward。
5. response 的平均 token entropy。
6. 平均 response 长度、正确 responses 的平均 response 长度，以及错误 responses 的平均 response 长度。

**问题** (`log_generations`)：Logging generations (1 分)

交付物：实现一个 `log_generations` 函数，可用于记录模型生成结果。

## 4.3 SFT 实验

使用上面的组件，你现在将实现完整的 SFT 过程（算法 1），在 MATH 数据集上微调 Qwen 2.5 Math 1.5B Base 模型。`/data/a5-alignment/MATH/sft.jsonl` 中的每个样例包含一个 formatted prompt 和一个 target response，其中 target response 包含思维链推理轨迹和最终答案。具体来说，每个样例是一个 JSON 元素，类型为 `{"prompt": str, "response": str}`。

为了跟踪模型在训练过程中的进展，你应当周期性地在 MATH validation set 上评估它。运行脚本时应使用 2 个 GPU，其中一个 GPU 用于 policy model，另一个用于 vLLM instance 来评估 policy。为了让这工作，下面是一些 starter code，用于初始化 vLLM，并在每个 rollout phase 之前将 policy weights 加载到 vLLM instance 中：

```
from vllm.model_executor import set_random_seed as vllm_set_random_seed

def init_vllm(model_id: str, device: str, seed: int, gpu_memory_utilization: float = 0.85):
    """
    Start the inference process, here we use vLLM to hold a model on
    a GPU separate from the policy.
    """
    vllm_set_random_seed(seed)
    # Monkeypatch from TRL:
    # https://github.com/huggingface/trl/blob/
    # 22759c820867c8659d00082ba8cf004e963873c1/trl/trainer/grpo_trainer.py
    # Patch vLLM to make sure we can
    # (1) place the vLLM model on the desired device (world_size_patch) and
    # (2) avoid a test that is not designed for our setting (profiling_patch).
    world_size_patch = patch("torch.distributed.get_world_size", return_value=1)
    profiling_patch = patch(
        "vllm.worker.worker.Worker._assert_memory_footprint_increased_during_profiling",
        return_value=None
    )
    with world_size_patch, profiling_patch:
        return LLM(
            model=model_id,
            device=device,
            dtype=torch.bfloat16,
            enable_prefix_caching=True,
            gpu_memory_utilization=gpu_memory_utilization,
        )

def load_policy_into_vllm_instance(policy: PreTrainedModel, llm: LLM):
    """
    Copied from https://github.com/huggingface/trl/blob/
    22759c820867c8659d00082ba8cf004e963873c1/trl/trainer/grpo_trainer.py#L670.
    """
```

```
state_dict = policy.state_dict()
llm_model = llm.llm_engine.model_executor.driver_worker.model_runner.model
llm_model.load_weights(state_dict.items())
```

你可能会发现，同时针对 train 和 validation steps 记录指标很有帮助（这在后续 RL 实验中也很有用）。要在 wandb 中实现，可以使用如下代码：

```
# Setup wandb metrics
wandb.define_metric("train_step") # the x-axis for training
wandb.define_metric("eval_step") # the x-axis for evaluation

# everything that starts with train/ is tied to train_step
wandb.define_metric("train/*", step_metric="train_step")

# everything that starts with eval/ is tied to eval_step
wandb.define_metric("eval/*", step_metric="eval_step")
```

最后，我们建议使用 gradient clipping, clip value 为 1.0。

**问题** (sft\_experiment)：在 MATH 数据集上运行 SFT（2 分）（2 H100 小时）

1. 使用 Qwen 2.5 Math 1.5B base model，在提供的 reasoning SFT 样例（/data/a5-alignment/MATH/sft.jsonl）上运行 SFT，改变用于 SFT 的 unique examples 数量，范围为 {128, 256, 512, 1024}，并且使用 full dataset。调节 learning rate 和 batch size，在使用 full dataset 时达到至少 15% validation accuracy。

交付物：与不同 dataset sizes 相关的 validation accuracy curves。

2. 过滤 reasoning SFT 样例，只保留能产生正确答案的样例。在完整的过滤后数据集上运行 SFT，并报告过滤后数据集大小以及你达到的 validation accuracy。

交付物：报告数据集大小和你获得的 validation accuracy curve。将你的发现与前一个 SFT 实验进行比较。

## 5 MATH 的专家迭代

在上一节中，我们观察到，通过过滤掉 SFT 数据中的坏样例，可以提升 SFT 模型表现。本节将更进一步：我们会把这种过滤过程应用到基础模型自己生成的推理轨迹上。这个过程在文献中称为专家迭代（expert iteration）[Anthony et al., 2017]，在语言模型语境下已有 Cobbe et al. [2021b]、Zelikman et al. [2022]、Dohan et al. [2022]、Gulcehre et al. [2023] 等工作探索。

算法 2 专家迭代 (EI)

输入：初始策略模型  $\pi_{\theta_{init}}$ ；奖励函数  $R$ ；任务问题  $D$

1. 策略模型  $\pi_{\theta} \leftarrow \pi_{\theta_{init}}$
2. 对  $step = 1, \dots, n_{ei\_steps}$ ：
3. 从  $D$  中采样一批问题  $D_b$
4. 设置旧策略模型  $\pi_{\theta_{old}} \leftarrow \pi_{\theta}$
5. 对  $D_b$  中每个问题  $q$ ，采样  $G$  个输出  $\{o(i)\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot | q)$
6. 对每个采样输出  $o(i)$ ，通过运行奖励函数  $R(q, o(i))$  计算奖励  $\{r(i)\}_{i=1}^G$
7. 过滤掉错误输出（即  $r(i) = 0$  的  $o(i)$ ），得到由正确 question-response pairs 组成的数据集  $D_{sft}$
8.  $\pi_{\theta} \leftarrow SFT(\pi_{\theta}, D_{sft})$ （算法 1）
9. 结束循环

输出： $\pi_{\theta}$

接下来，我们将在 MATH 数据集上运行专家迭代。

作为提示，你应当向 vLLM 的 `SamplingParams` 传入 `min_tokens` 值，这可以确保不会生成空字符串（空字符串可能会导致下游实现中出现 NaN）。可以这样做：

```
sampling_min_tokens = 4
sampling_params = SamplingParams(
    temperature=sampling_temperature,
    max_tokens=sampling_max_tokens,
    min_tokens=sampling_min_tokens,
    n=G,
    seed=seed,
)
```

与 SFT 中一样，你应使用 gradient clipping，clip value 为 1.0。

**问题** (expert\_iteration\_experiment)：在 MATH 数据集上运行专家迭代（2 分）（6 H100 小时）

使用 Qwen 2.5 Math 1.5B Base 模型，在 MATH 数据集（位于 `/data/a5-alignment/MATH/train.jsonl`）上运行专家迭代，改变每个问题的 rollout 数量 `G` 和 SFT step 中使用的 epochs 数，并使用 `n_ei_steps = 5`。改变每个专家迭代 step 的 batch size（即 `Db` 的大小），取值为 `{512, 1024, 2048}`。（你不需要尝试这些超参数的所有组合；只需要足以对每个因素得出结论即可。）记录训练过程中模型 response 的 entropy。确保让 vLLM 在第二个 answer tag `</answer>` 处终止 generation，如 SFT 一节中所做。

交付物：与不同 rollout configurations 相关的 validation accuracy curves。至少尝试 2 种不同的 rollout counts 和 epoch counts。

交付物：一个在 MATH 上达到至少 15% validation accuracy 的模型。

交付物：2 句简短讨论，与 SFT performance 进行比较，并比较 EI steps 之间的 performance。

交付物：模型 responses 的 entropy 随训练变化的 plot。

## 6 策略梯度入门

语言模型研究中的一个令人兴奋的新发现是：在可验证奖励上对强基础模型执行 RL，可以显著提升其推理能力和表现 [OpenAI et al., 2024, DeepSeek-AI et al., 2025]。最强的开放推理模型，如 DeepSeek R1 和 Kimi k1.5 [Team et al., 2025]，使用了策略梯度，这是一类强大的强化学习算法，可以优化任意奖励函数。

下面我们简要介绍语言模型上的 policy gradients。我们的介绍紧密参考了几份很棒的资源，它们对这些概念进行了更深入的讲解：OpenAI 的 Spinning Up in Deep RL [Achiam, 2018a] 和 Nathan Lambert 的 Reinforcement Learning from Human Feedback (RLHF) Book [Lambert, 2024]。

### 6.1 将语言模型视为策略

参数为  $\theta$  的 causal language model (LM) 定义了在当前文本前缀 `st`（状态/观察）下，下一个 token `at` in `V` 的概率分布。在 RL 语境中，我们把下一个 token `at` 视为一个 action，把当前文本前缀 `st` 视为 state。因此，LM 是一个 categorical stochastic policy：

$$at \sim \pi_{\theta}(\cdot \mid st), \pi_{\theta}(at \mid st) = [\text{softmax } f_{\theta}(st)]_{at}. \quad (3)$$

使用 policy gradients 优化 policy 时，需要两个基本操作：

1. 从 policy 中采样：从上述 categorical distribution 抽取 action `at`；

2. 给 action 的 log-likelihood 打分：评估  $\log \pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)$ 。

通常，在对 LLM 做 RL 时， $\mathbf{s}_t$  是目前为止产生的部分 completion/solution，而每个  $\mathbf{a}_t$  是 solution 的下一个 token；当发出 end-of-text token（如 `<|end_of_text|>`）或在我们的 `r1_zero` 提示词中发出 `</answer>` 时，episode 结束。

## 6.2 轨迹

一个（有限时域的）trajectory 是 agent 经历的 state 与 action 交错序列：

$$\tau = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T), \quad (4)$$

其中  $T$  是 trajectory 的长度，即  $\mathbf{a}_T$  是 end-of-text token，或者我们已经达到最大 generation token budget。

初始 state 从 start distribution 抽取， $\mathbf{s}_0 \sim p_0(\mathbf{s}_0)$ ；在 LLM 的 RL 中， $p_0(\mathbf{s}_0)$  是 formatted prompts 上的分布。一般设置中，state transitions 遵循某些环境 dynamics  $\mathbf{s}_{t+1} \sim P(\cdot \mid \mathbf{s}_t, \mathbf{a}_t)$ 。在 LLM 的 RL 中，环境是确定性的：下一个 state 是旧 prefix 与发出的 token 拼接而成， $\mathbf{s}_{t+1} = \mathbf{s}_t \parallel \mathbf{a}_t$ 。Trajectories 也称为 episodes 或 rollouts；我们会交替使用这些术语。

## 6.3 奖励与回报

标量奖励  $r_t = R(\mathbf{s}_t, \mathbf{a}_t)$  用于判断在 state  $\mathbf{s}_t$  采取 action 的即时质量。对于可验证领域上的 RL，标准做法是给中间步骤分配零奖励，并给 terminal action 分配可验证奖励：

$r_T = R(\mathbf{s}_T, \mathbf{a}_T) := 1$  if the trajectory  $\mathbf{s}_T \parallel \mathbf{a}_T$  matches the ground-truth according to our reward function, otherwise 0.

Return  $R(\tau)$  聚合 trajectory 上的奖励。两个常见选择是有限时域 undiscounted returns：

$$R(\tau) := \sum_{t=0}^{T-1} r_t, \quad (5)$$

以及无限时域 discounted returns：

$$R(\tau) := \sum_{t=0}^{\infty} \gamma^t r_t, \quad 0 < \gamma < 1. \quad (6)$$

在我们的情况中，由于 episodes 有自然终止点（end-of-text 或 max generation length），我们将使用 undiscounted formulation。

Agent 的目标是最大化 expected return：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)], \quad (7)$$

从而得到优化问题：

$$\theta^* = \arg \max_{\theta} J(\theta). \quad (8)$$

## 6.4 Vanilla Policy Gradient

接下来，让我们尝试用 expected return 上的 gradient ascent 来学习 policy parameters  $\theta$ ：

$$\theta_{k+1} = \theta_k + \alpha \nabla_{\theta} J(\theta_k). \quad (9)$$

我们将使用的核心恒等式是 REINFORCE policy gradient，如下所示：

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t) R(\tau) \right]. \quad (10)$$

推导 policy gradient。

这个方程是如何得到的？为完整起见，我们给出该恒等式的推导。我们将使用几个恒等式。

1. trajectory 的概率为：

$$P(\tau | \theta) = p_{\theta}(s_0) \prod_{t=0}^{T-1} P(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t). \quad (11)$$

因此，trajectory 的 log-probability 是：

$$\log P(\tau | \theta) = \log p_{\theta}(s_0) + \sum_{t=0}^{T-1} [\log P(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t)]. \quad (12)$$

2. log-derivative trick：

$$\nabla_{\theta} P = P \nabla_{\theta} \log P. \quad (13)$$

3. 环境项对  $\theta$  是常数。 $p_{\theta}$ 、 $P(\cdot | \cdot)$  和  $R(\tau)$  不依赖于 policy parameters，因此：

$$\nabla_{\theta} p_{\theta} = \nabla_{\theta} P = \nabla_{\theta} R(\tau) = 0. \quad (14)$$

应用以上事实：

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} E_{\{\tau \sim \pi_{\theta}\}}[R(\tau)] \quad (15)$$

$$= \nabla_{\theta} \sum_{\tau} P(\tau | \theta) R(\tau) \quad (16)$$

$$= \sum_{\tau} \nabla_{\theta} P(\tau | \theta) R(\tau) \quad (17)$$

$$= \sum_{\tau} P(\tau | \theta) \nabla_{\theta} \log P(\tau | \theta) R(\tau) \quad (\text{Log-derivative trick}) \quad (18)$$

$$= E_{\{\tau \sim \pi_{\theta}\}}[\nabla_{\theta} \log P(\tau | \theta) R(\tau)], \quad (19)$$

因此，将 trajectory 的 log-probability 代入，并利用环境项为常数的事实，我们得到 vanilla 或 REINFORCE policy gradient：

$$\nabla_{\theta} J(\pi_{\theta}) = E_{\{\tau \sim \pi_{\theta}\}}[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau)]. \quad (20)$$

直观上，该梯度会提高高 return trajectory 中每个 action 的 log probability，并降低其他情况的 log probability。

梯度的样本估计。给定一批  $N$  个 rollouts  $D = \{\tau(i)\}_{i=1}^N$ ，这些 rollouts 是通过采样起始 state  $s_0(i) \sim p_{\theta}(s_0)$  并在环境中运行 policy  $\pi_{\theta}$  收集的，我们可以形成如下无偏梯度估计：

$$\hat{g} = (1/N) \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t(i) | s_t(i)) R(\tau(i)). \quad (21)$$

该向量用于 gradient-ascent update  $\theta \leftarrow \theta + \alpha \hat{g}$ 。

## 6.5 Policy Gradient Baselines

Vanilla policy gradient 的主要问题是梯度估计方差很高。一个常见的缓解技术是从奖励中减去一个只依赖于 state 的 baseline function  $b$ 。这是一类 control variate [Ross, 2022]：其思想是在不引入 bias 的前提下，通过减去一个与估计量相关的项来降低方差。

令 baselined policy gradient 为：

$$B = E_{\{\tau \sim \pi_{\theta}\}}[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (R(\tau) - b(s_t))]. \quad (22)$$

作为例子，一个合理的 baseline 是 on-policy value function  $V^{\pi}(s) = E_{\{\tau \sim \pi_{\theta}\}}[R(\tau) | s_t = s]$ ，即如果从  $s_t = s$  开始并遵循 policy  $\pi_{\theta}$ ，expected return 是多少。那么  $R(\tau) - V^{\pi}(s_t)$  直观上表示实际 trajectory 比预期好多少。

只要 baseline 只依赖于 state，baselined policy gradient 就是无偏的。我们可以将其重写为：

$$B = E_{\{\tau \sim \pi_{\theta}\}}[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau)] - E_{\{\tau \sim \pi_{\theta}\}}[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t)]. \quad (23)$$

聚焦 baseline 项，可得：

$$E_{\{\tau \sim \pi_{\theta}\}}[\sum_{\{t=0\}^{\{T\}} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t)] = \sum_{\{t=0\}^{\{T\}} E_{\{s_t\}}[b(s_t) E_{\{a_t \sim \pi_{\theta}(\cdot | s_t)\}}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]]]. \quad (24)$$

一般而言，score function 的期望为零： $E_{\{x \sim P_{\theta}\}}[\nabla_{\theta} \log P_{\theta}(x)] = 0$ 。因此，式 (24) 中的表达式为零，并且：

$$B = E_{\{\tau \sim \pi_{\theta}\}}[\sum_{\{t=0\}^{\{T\}} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau)] - 0 = \nabla_{\theta} J(\pi_{\theta}), \quad (25)$$

所以 baselined policy gradient 是无偏的。后续我们会运行一个实验，观察 baselining 是否提升下游性能。

关于 policy gradient “losses” 的说明。当我们在 PyTorch 之类的框架中实现 policy gradient 方法时，会定义一个所谓的 policy gradient loss `pg_loss`，使得调用 `pg_loss.backward()` 会将我们模型参数的 gradient buffers 填充为近似 policy gradient `g_hat`。数学上可写为：

$$\text{pg\_loss} = (1/N) \sum_{\{i=1\}^{\{N\}} \sum_{\{t=0\}^{\{T\}} \log \pi_{\theta}(a_t(i) | s_t(i)) (R(\tau(i)) - b(s_t(i)))}. \quad (26)$$

`pg_loss` 并不是规范意义上的 loss——把 `pg_loss`

作为训练集或验证集上的评估指标来报告并没有意义，一个好的 validation `pg_loss`

并不表示模型泛化良好。`pg_loss` 实际上只是一个标量，使得当我们调用 `pg_loss.backward()` 时，通过反向传播得到的梯度就是近似 policy gradient `g_hat`。

做 RL 时，你应该始终记录和报告 train 与 validation

rewards。这些才是“有意义”的评估指标，也是我们试图用 policy gradient 方法优化的东西。

## 6.6 Off-Policy Policy Gradient

REINFORCE 是一个 on-policy 算法：训练数据由我们正在优化的同一个 policy 收集。可以将 REINFORCE 算法写成：

1. 从当前 policy  $\pi_{\theta}$  采样一批 rollouts  $\{\tau(i)\}^{N}_{i=1}$ 。
2. 近似 policy gradient： $\nabla_{\theta} J(\pi_{\theta}) \approx \mathbf{g\_hat} = (1/N) \sum_i \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t(i) | s_t(i)) R(\tau(i))$ 。
3. 使用计算得到的梯度更新 policy parameters： $\theta \leftarrow \theta + \alpha \mathbf{g\_hat}$ 。

我们需要进行大量 inference 来采样一批新的 rollouts，却只执行一个 gradient step。LM 的行为通常不会在单步中显著改变，因此这种 on-policy 方法效率很低。

Off-policy policy gradient。在 off-policy learning 中，我们的 rollouts 由正在优化的 policy 之外的某个 policy 采样。PPO 和 GRPO 等流行 policy gradient 算法的 off-policy 变体，使用来自旧版本 policy  $\pi_{\theta_{old}}$  的 rollouts 来优化当前 policy  $\pi_{\theta}$ 。Off-policy policy gradient 估计为：

$$\mathbf{g\_hat\_off-policy} = (1/N) \sum_{\{i=1\}^{\{N\}} \sum_{\{t=0\}^{\{T\}} [\pi_{\theta}(a_t(i) | s_t(i)) / \pi_{\theta_{old}}(a_t(i) | s_t(i))] \nabla_{\theta} \log \pi_{\theta}(a_t(i) | s_t(i)) R(\tau(i))}. \quad (27)$$

这看起来像 vanilla policy gradient 的 importance sampled 版本，其中重加权项为  $\pi_{\theta}(a_t(i) | s_t(i)) / \pi_{\theta_{old}}(a_t(i) | s_t(i))$ 。

的确，式 (27) 可以通过 importance sampling 推导，并应用一个近似；只要  $\pi_{\theta}$  和  $\pi_{\theta_{old}}$  相差不太大，该近似就是合理的。更多内容见 Degris et al. [2013]。

## 7 Group Relative Policy Optimization

接下来，我们将描述 Group Relative Policy Optimization (GRPO)，这是你将实现并用于数学问题求解实验的 policy gradient 变体。

## 7.1 GRPO 算法

Advantage estimation。GRPO 的核心思想是：从 policy  $\pi_\theta$  为每个问题采样多个 outputs，并用它们计算 baseline。这很方便，因为我们避免了学习神经 value function  $V_\phi(s)$  的需要；value function 可能难以训练，并且从系统角度看很繁琐。

对于一个问题  $q$  和一组 outputs  $\{o(i)\}_{i=1}^G \sim \pi_\theta(\cdot | q)$ ，令  $r(i) = R(q, o(i))$  为第  $i$  个 output 的 reward。DeepSeekMath [Shao et al., 2024] 和 DeepSeek R1 [DeepSeek-AI et al., 2025] 将第  $i$  个 output 的 group-normalized reward 计算为：

$$A(i) = (r(i) - \text{mean}(r(1), r(2), \dots, r(G))) / (\text{std}(r(1), r(2), \dots, r(G)) + \text{advantage\_eps}), \quad (28)$$

其中 `advantage_eps` 是一个小常数，用于避免除以零。注意，该 advantage  $A(i)$  对 response 中每个 token 都相同，即  $A_t(i) = A(i)$ ， $\forall t$  in  $1, \dots, |o(i)|$ ，因此下文省略  $t$  下标。

高层算法。在深入 GRPO objective 之前，我们先通过 Shao et al. [2024] 的算法 3 了解训练循环的整体结构。

算法 3 Group Relative Policy Optimization (GRPO)

输入：初始策略模型  $\pi_{\theta_{\text{init}}}$ ；奖励函数  $R$ ；任务问题  $D$

1. 策略模型  $\pi_\theta \leftarrow \pi_{\theta_{\text{init}}}$
2. 对  $\text{step} = 1, \dots, n_{\text{grpo\_steps}}$ ：
3. 从  $D$  中采样一批问题  $D_b$
4. 设置旧策略模型  $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
5. 对  $D_b$  中每个问题  $q$ ，采样  $G$  个输出  $\{o(i)\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$
6. 对每个采样输出  $o(i)$ ，通过运行奖励函数  $R(q, o(i))$  计算奖励  $\{r(i)\}_{i=1}^G$
7. 使用 group normalization 计算  $A(i)$  (式 28)
8. 对  $\text{train\_step} = 1, \dots, n_{\text{train\_steps\_per\_rollout\_batch}}$ ：
9. 通过最大化 GRPO-Clip objective (稍后讨论，式 29) 更新 policy model  $\pi_\theta$
10. 结束内层循环
11. 结束外层循环

输出： $\pi_\theta$

GRPO objective。GRPO objective 结合了三个想法：

1. Off-policy policy gradient，如式 (27)。
2. 使用 group normalization 计算 advantages  $A(i)$ ，如式 (28)。
3. 类似 Proximal Policy Optimization (PPO, Schulman et al. [2017]) 中的 clipping 机制。

Clipping 的目的是：当在单个 rollout batch 上执行多次 gradient steps 时保持稳定。它通过防止 policy  $\pi_\theta$  远离旧 policy 来工作。

完整 GRPO-Clip objective 可写为：

$$J_{\text{GRPO-Clip}}(\theta) = E_{\{q \sim D, \{o(i)\} \sim \pi_{\theta}(\cdot | q)\}} \left[ \left( \frac{1}{G} \sum_i \frac{1}{|o(i)|} \right) \sum_t \min(\text{ratio}_t(i) A(i), \text{clip}(\text{ratio}_t(i), 1-\epsilon, 1+\epsilon) A(i)) \right], \quad (29)$$

其中：

$$\text{ratio}_t(i) = \pi_{\theta}(o_t(i) | q, o_{<t}(i)) / \pi_{\theta_{\text{old}}}(o_t(i) | q, o_{<t}(i)).$$

超参数  $\epsilon > 0$  控制 policy 可以改变多少。为了理解这一点，我们可以跟随 Achiam [2018a,b] 将 per-token objective 以更直观的方式重写。定义函数：

$$g(\epsilon, A(i)) = (1 + \epsilon)A(i) \text{ if } A(i) \geq 0, \text{ otherwise } (1 - \epsilon)A(i). \quad (30)$$

于是 per-token objective 可重写为：

$$\text{per-token objective} = \min(\text{ratio}_t(i) A(i), g(\epsilon, A(i))).$$

现在可以分情况讨论。当 advantage  $A(i)$  为正时，per-token objective 简化为：

$$\text{per-token objective} = \min(\text{ratio}_t(i), 1 + \epsilon) A(i).$$

由于  $A(i) > 0$ ，如果 action  $o_t(i)$  在  $\pi_{\theta}$  下变得更可能，即  $\pi_{\theta}(o_t(i) | q, o_{<t}(i))$  增加，则 objective 上升。使用 min 的 clipping 限制了 objective 可以上升多少：一旦  $\pi_{\theta}(o_t(i) | q, o_{<t}(i)) > (1 + \epsilon) \pi_{\theta_{\text{old}}}(o_t(i) | q, o_{<t}(i))$ ，该 per-token objective 就达到最大值  $(1 + \epsilon)A(i)$ 。因此，policy  $\pi_{\theta}$  不会被激励得离旧 policy  $\pi_{\theta_{\text{old}}}$  太远。

类似地，当 advantage  $A(i)$  为负时，模型会试图降低  $\pi_{\theta}(o_t(i) | q, o_{<t}(i))$ ，但不会被激励到低于  $(1 - \epsilon)\pi_{\theta_{\text{old}}}(o_t(i) | q, o_{<t}(i))$ （完整论证见 Achiam [2018b]）。

## 7.2 实现

既然我们已经从高层理解了 GRPO 训练循环和 objective，接下来开始实现其中的组件。SFT 和 EI 章节中实现的许多组件也会在 GRPO 中复用。

计算 advantages (group-normalized rewards)。首先，我们将实现对 rollout batch 中每个样例计算 advantages 的逻辑，即 group-normalized rewards。我们将考虑两种获得 group-normalized rewards 的方法：上面式 (28) 中的方法，以及近期提出的简化方法。

Dr. GRPO [Liu et al., 2025] 指出，通过  $\text{std}(r(1), r(2), \dots, r(G))$  归一化会奖励 batch 中答案正确性变化较小的问题，这可能并不理想。他们提出简单移除 normalization step，计算：

$$A(i) = r(i) - \text{mean}(r(1), r(2), \dots, r(G)). \quad (31)$$

我们将实现两种变体，并在后续作业中比较其性能。

**问题** (compute\_group\_normalized\_rewards)：Group normalization (2 分)

交付物：实现 `compute_group_normalized_rewards` 方法，该方法为每个 rollout response 计算 raw rewards，在其组内进行归一化，并返回 normalized 和 raw rewards，以及你认为有用的元数据。

推荐接口如下：

```
def compute_group_normalized_rewards(
    reward_fn,
    rollout_responses,
    repeated_ground_truths,
    group_size,
    advantage_eps,
    normalize_by_std,
):
```

为每个 rollout response 组计算 rewards，并按 group size 归一化。

参数：

- `reward_fn`: `Callable[[str, str], dict[str, float]]`：根据 ground truth 给 rollout responses 打分，生成包含 "reward"、"format\_reward" 和 "answer\_reward" 的字典。
- `rollout_responses`: `list[str]`：policy 的 rollouts。该列表长度为 `rollout_batch_size = n_prompts_per_rollout_batch * group_size`。
- `repeated_ground_truths`: `list[str]`：样例的 ground truths。该列表长度为 `rollout_batch_size`，因为每个样例的 ground truth 重复 `group_size` 次。
- `group_size`: `int`：每个问题的 responses 数量（group）。
- `advantage_eps`: `float`：防止 normalization 中除以零的小常数。
- `normalize_by_std`: `bool`：如果为 True，则除以每组标准差；否则只减去 group mean。

返回：

`tuple[torch.Tensor, torch.Tensor, dict[str, float]]`。

- `advantages`：形状为 `(rollout_batch_size,)`。每个 rollout response 的 group-normalized rewards。
- `raw_rewards`：形状为 `(rollout_batch_size,)`。未归一化 rewards。
- `metadata`：你选择记录的其他统计量（例如 rewards 的 mean、std、max/min）。

为了测试你的代码，实现 `[adapters.run_compute_group_normalized_rewards]`。然后运行 `uv run pytest -k test_compute_group_normalized_rewards`，确保通过。

Naive policy gradient loss。接下来，我们将实现一些用于计算“losses”的方法。

再次提醒/声明，这些并不是真正规范意义上的 loss，不应该作为评估指标报告。在 RL 中，你应当跟踪 train 和 validation returns，以及其他指标（见 § 6.5 的讨论）。

我们将从 naive policy gradient loss 开始，它只是将 advantage 与 actions 的 log-probability 相乘（并取负）。对于 question `q`、response `o` 和 response token `ot`，naive per-token policy gradient loss 为：

$$- A_t \cdot \log p_{\theta}(ot \mid q, o_{<t}). \quad (32)$$

问题（`compute_naive_policy_gradient_loss`）：Naive policy gradient（1 分）

交付物：实现 `compute_naive_policy_gradient_loss`，它使用 raw rewards 或预先计算的 advantages 计算 per-token policy-gradient loss。

推荐接口如下：

```
def compute_naive_policy_gradient_loss(
    raw_rewards_or_advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
) -> torch.Tensor:
```

计算每个 token 的 policy-gradient loss，其中 `raw_rewards_or_advantages` 可以是 raw reward，也可以是已经归一化的 advantage。

参数：

- `raw_rewards_or_advantages`: `torch.Tensor`：形状为 `(batch_size, 1)`，每个 rollout response 的标量 reward/advantage。
- `policy_log_probs`: `torch.Tensor`：形状为 `(batch_size, sequence_length)`，每个 token 的 logprobs。

返回：

- `torch.Tensor`：形状为  $(\text{batch\_size}, \text{sequence\_length})$ ，per-token policy-gradient loss（将在训练循环中跨 batch 和 sequence 维度聚合）。

实现提示：

- 将 `raw_rewards_or_advantages` broadcast 到 `sequence_length` 维度。

为了测试你的代码，实现 `[adapters.run_compute_naive_policy_gradient_loss]`。然后运行 `uv run pytest -k test_compute_naive_policy_gradient_loss` 并确保通过。

GRPO-Clip loss。接下来，我们将实现更有趣的 GRPO-Clip loss。

Per-token GRPO-Clip loss 为：

$$- \min( [\pi\theta(o_t|q, o < t) / \pi\theta_{old}(o_t|q, o < t)] A_t, \text{clip}([\pi\theta(o_t|q, o < t) / \pi\theta_{old}(o_t|q, o < t)], 1-\epsilon, 1+\epsilon) A_t ). \quad (33)$$

**问题** (`compute_grpo_clip_loss`)：GRPO-Clip loss（2 分）

交付物：实现 `compute_grpo_clip_loss`，计算 per-token GRPO-Clip loss。

推荐接口如下：

```
def compute_grpo_clip_loss(
    advantages: torch.Tensor,
    policy_log_probs: torch.Tensor,
    old_log_probs: torch.Tensor,
    cliprange: float,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

参数：

- `advantages: torch.Tensor`：形状为  $(\text{batch\_size}, 1)$ ，每个样例的 advantages  $A_o$ 。
- `policy_log_probs: torch.Tensor`：形状为  $(\text{batch\_size}, \text{sequence\_length})$ ，来自正在训练的 policy 的 per-token log probs。
- `old_log_probs: torch.Tensor`：形状为  $(\text{batch\_size}, \text{sequence\_length})$ ，来自旧 policy 的 per-token log probs。
- `cliprange: float`：clip 参数  $\epsilon$ （例如 0.2）。

返回：

`tuple[torch.Tensor, dict[str, torch.Tensor]]`。

- `loss`：形状为  $(\text{batch\_size}, \text{sequence\_length})$  的 `torch.Tensor`，per-token clipped loss。
- `metadata`：包含你希望记录的任何内容。我们建议记录每个 token 是否被 clipped，即 RHS 中 clipped policy gradient loss 是否低于 LHS。

实现提示：

- 将 `advantages` broadcast 到 `sequence_length`。

为了测试你的代码，实现 `[adapters.run_compute_grpo_clip_loss]`。然后运行 `uv run pytest -k test_compute_grpo_clip_loss` 并确保通过。

Policy gradient loss wrapper。我们将运行 ablations，比较三种不同版本的 policy gradient：

(a) `no_baseline`：没有 baseline 的 naive policy gradient loss，即 advantage 只是 raw rewards  $A = R(q, o)$ 。

(b) `reinforce_with_baseline` : naive policy gradient loss, 但使用 group-normalized rewards 作为 advantage。如果 `r_bar` 是来自 `compute_group_normalized_rewards` 的 group-normalized rewards (可能按 group standard deviation 归一化, 也可能没有), 则  $A = r\_bar$ 。

(c) `grpo_clip` : GRPO-Clip loss。

为方便起见, 我们将实现一个 wrapper, 以便在这三种 policy gradient losses 之间轻松切换。

**问题** (`compute_policy_gradient_loss`) : Policy-gradient wrapper (1 分)

交付物: 实现 `compute_policy_gradient_loss`, 这是一个 convenience wrapper, 分发到正确的 loss routine (`no_baseline`、`reinforce_with_baseline` 或 `grpo_clip`), 并返回 per-token loss 和任何 auxiliary statistics。

推荐接口如下:

```
def compute_policy_gradient_loss(
    policy_log_probs: torch.Tensor,
    loss_type: Literal["no_baseline", "reinforce_with_baseline", "grpo_clip"],
    raw_rewards: torch.Tensor | None = None,
    advantages: torch.Tensor | None = None,
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

选择并计算所需的 policy-gradient loss。

参数:

- `policy_log_probs` (`batch_size`, `sequence_length`): 来自正在训练的 policy 的 per-token log-probabilities。
- `loss_type`: "no\_baseline"、"reinforce\_with\_baseline" 或 "grpo\_clip" 之一。
- `raw_rewards`: 当 `loss_type == "no_baseline"` 时必需; 形状为 (`batch_size`, 1)。
- `advantages`: 对于 "reinforce\_with\_baseline" 和 "grpo\_clip" 必需; 形状为 (`batch_size`, 1)。
- `old_log_probs`: 对 "grpo\_clip" 必需; 形状为 (`batch_size`, `sequence_length`)。
- `cliprange`: 对 "grpo\_clip" 必需; 用于 clipping 的标量  $\epsilon$ 。

返回:

`tuple[torch.Tensor, dict[str, torch.Tensor]]`。

- `loss` (`batch_size`, `sequence_length`): per-token loss。
- `metadata`: 来自底层 routine 的统计量 (例如 GRPO-Clip 的 clip fraction)。

实现提示:

- 委托给 `compute_naive_policy_gradient_loss` 或 `compute_grpo_clip_loss`。
- 执行参数检查 (参考上面的 assertion pattern)。
- 将返回的 metadata 聚合到一个字典中。

为了测试你的代码, 实现 [`adapters.run_compute_policy_gradient_loss`]。然后运行 `uv run pytest -k test_compute_policy_gradient_loss` 并验证通过。

Masked mean。到目前为止, 我们已经拥有计算 advantages、log probabilities、per-token losses 以及有用统计量 (如 per-token entropies 和 clip fractions) 所需的逻辑。为了将形状为 (`batch_size`,

`sequence_length`) 的 per-token loss tensors 缩减为一个 loss 向量（每个样例一个标量），我们会在 sequence 维度上计算 loss 的均值，但只在 response 对应的索引上计算（即 `mask[i, j] == 1` 的 token positions）。

按 sequence length 归一化是大多数 LLM RL

代码库中的常见做法，但这并不明显就是正确选择——你可能会注意到，在式 (21) 的 policy gradient estimate 中，并没有  $1/T(i)$  的归一化因子。我们将从这个标准 primitive 开始，它通常被称为 `masked_mean`，但后面会测试使用 SFT 中实现的 `masked_normalize` 方法。

我们允许指定计算均值的维度；如果 `dim` 为 `None`，则对所有 masked elements 计算均值。这可能有助于获得 response tokens 上的 average per-token entropies、clip fractions 等。

**问题** (`masked_mean`)：Masked mean (1 分)

交付物：实现 `masked_mean` 方法，它在尊重布尔 mask 的情况下对 tensor 元素求平均。

推荐接口如下：

```
def masked_mean(
    tensor: torch.Tensor,
    mask: torch.Tensor,
    dim: int | None = None,
) -> torch.Tensor:
```

沿给定维度计算 tensor 的均值，只考虑 `mask == 1` 的元素。

参数：

- `tensor: torch.Tensor`：要被平均的数据。
- `mask: torch.Tensor`：与 tensor 形状相同；值为 1 的位置会被纳入均值。
- `dim: int | None`：计算均值的维度。如果为 `None`，则在所有 masked elements 上计算均值。

返回：

- `torch.Tensor`：masked mean；形状遵循 `tensor.mean(dim)` 语义。

为了测试你的代码，实现 `[adapters.run_masked_mean]`。然后运行 `uv run pytest -k test_masked_mean` 并确保通过。

GRPO microbatch train step。现在我们已经准备好实现 GRPO 的单个 microbatch train step（回忆一下，如果 `gradient_accumulation_steps > 1`，对于一个 train minibatch，我们会迭代多个 microbatches）。

具体来说，给定 raw rewards 或 advantages 以及 log probs，我们将计算 per-token loss，使用 `masked_mean` 聚合为每个样例一个标量 loss，在 batch 维度上取平均，针对 gradient accumulation 做调整，并反向传播。

**问题** (`grpo_microbatch_train_step`)：Microbatch train step (3 分)

交付物：实现 GRPO 的单个 micro-batch update，包括 policy-gradient loss、使用 mask 求平均，以及 gradient scaling。

推荐接口如下：

```
def grpo_microbatch_train_step(
    policy_log_probs: torch.Tensor,
    response_mask: torch.Tensor,
    gradient_accumulation_steps: int,
    loss_type: Literal["no_baseline", "reinforce_with_baseline", "grpo_clip"],
    raw_rewards: torch.Tensor | None = None,
    advantages: torch.Tensor | None = None,
    old_log_probs: torch.Tensor | None = None,
    cliprange: float | None = None,
) -> tuple[torch.Tensor, dict[str, torch.Tensor]]:
```

在一个 microbatch 上执行 forward-and-backward pass。

参数：

- `policy_log_probs (batch_size, sequence_length)`：来自正在训练的 policy 的 per-token log-probabilities。
- `response_mask (batch_size, sequence_length)`：response token 为 1，prompt/padding 为 0。
- `gradient_accumulation_steps`：每个 optimizer step 的 microbatches 数。
- `loss_type`："no\_baseline"、"reinforce\_with\_baseline" 或 "grpo\_clip" 之一。
- `raw_rewards`：当 `loss_type == "no_baseline"` 时需要；形状为 `(batch_size, 1)`。
- `advantages`：当 `loss_type != "no_baseline"` 时需要；形状为 `(batch_size, 1)`。
- `old_log_probs`：GRPO-Clip 必需；形状为 `(batch_size, sequence_length)`。
- `cliprange`：GRPO-Clip 的 clip 参数  $\epsilon$ 。

返回：

`tuple[torch.Tensor, dict[str, torch.Tensor]]`。

- `loss`：标量 tensor。microbatch loss，已针对 gradient accumulation 调整。返回它是为了记录。
- `metadata`：来自底层 loss 调用的元数据，以及你希望记录的其他统计量。

实现提示：

- 你应当在该函数中调用 `loss.backward()`。确保对 gradient accumulation 做相应调整。

为了测试你的代码，实现 `[adapters.run_grpo_microbatch_train_step]`。然后运行 `uv run pytest -k test_grpo_microbatch_train_step` 并确认通过。

整合：GRPO train loop。现在我们将把完整的 GRPO train loop 组合起来。你应参考 § 7.1 的算法，使用我们已经实现的方法完成整体结构。

下面提供一些 starter hyperparameters。如果实现正确，使用这些参数应当能看到合理结果。

```
n_grpo_steps: int = 200
learning_rate: float = 1e-5
advantage_eps: float = 1e-6
rollout_batch_size: int = 256
group_size: int = 8
sampling_temperature: float = 1.0
sampling_min_tokens: int = 4 # As in Expiter, disallow empty string responses
sampling_max_tokens: int = 1024
epochs_per_rollout_batch: int = 1 # On-policy
train_batch_size: int = 256 # On-policy
gradient_accumulation_steps: int = 128 # microbatch size is 2, will fit on H100
gpu_memory_utilization: float = 0.85
loss_type: Literal[
    "no_baseline",
    "reinforce_with_baseline",
    "grpo_clip",
] = "reinforce_with_baseline"
use_std_normalization: bool = True
optimizer = torch.optim.AdamW(
    policy.parameters(),
    lr=learning_rate,
    weight_decay=0.0,
    betas=(0.9, 0.95),
)
```

这些默认超参数会让你从 on-policy 设置开始——对于每个 rollout batch，我们只执行一个 gradient step。以超参数来说，这意味着 `train_batch_size` 等于 `rollout_batch_size`，并且 `epochs_per_rollout_batch` 等于 1。

下面是一些 sanity check asserts 和 constants，它们应能避免一些边界情况，并指引你走向正确方向：

```
assert train_batch_size % gradient_accumulation_steps == 0, (  
    "train_batch_size must be divisible by gradient_accumulation_steps"  
)  
micro_train_batch_size = train_batch_size // gradient_accumulation_steps  
  
assert rollout_batch_size % group_size == 0, (  
    "rollout_batch_size must be divisible by group_size"  
)  
n_prompts_per_rollout_batch = rollout_batch_size // group_size  
  
assert train_batch_size >= group_size, (  
    "train_batch_size must be greater than or equal to group_size"  
)  
n_microbatches_per_rollout_batch = rollout_batch_size // micro_train_batch_size
```

其他提示：

- 记得使用 `r1_zero` prompt，并指示 vLLM 在第二个 `answer` tag `</answer>` 处停止 generation，如前面实验中一样。
- 建议使用 typer 做参数解析。
- 使用 gradient clipping，clip value 为 1.0。
- 你应当定期记录 validation rewards（例如每 5 或 10 步）。为了比较超参数，应至少在 1024 个 validation examples 上评估，因为 CoT/RL evaluations 可能比较 noisy。
- 按照我们的 losses 实现，GRPO-Clip 应只在 off-policy 时使用（因为它需要 old log-probabilities）。
- 在多个 epochs 的 gradient updates per rollout batch 的 off-policy 设置中，为每个 epoch 重新计算 old log-probabilities 会很浪费。相反，可以在每次 rollout batch generation phase 后计算一次 old log-probabilities，并在每个 epoch 重用。
- 不对 old log-probabilities 求导。
- 每次 optimizer update 时，你应记录以下部分或全部内容：

- loss。 - gradient norm。 - token entropy。 - 如果 off-policy，记录 clip fraction。 - train rewards（total、format 和 answer）。 - 任何你认为有助于调试的其他内容。

**问题** (grpo\_train\_loop)：GRPO train loop（5 分）

交付物：实现完整的 GRPO train loop。开始在 MATH 上训练 policy，并确认你看到 validation rewards 提升，同时 rollouts 随时间变化是合理的。提供 validation rewards 随 steps 变化的 plot，以及训练过程中若干 example rollouts。

## 8 GRPO 实验

现在我们可以开始实验 GRPO train loop，尝试不同超参数和算法修改。每个实验需要 2 个 GPU，一个用于 vLLM instance，另一个用于 policy。

关于提前停止 runs 的说明。如果你在 200 个 GRPO steps 之前看到超参数之间存在显著差异（例如某个配置发散，或明显次优），当然可以提前停止实验，把时间和算力留给后续 runs。下面提到的 GPU 小时只是粗略估计。

**问题** (grpo\_learning\_rate)：调节 learning rate（2 分）（6 H100 小时）

从上面建议的超参数开始，对 learning rates 做 sweep，并报告最终 validation answer rewards（如果 optimizer 发散，也请注明 divergence）。

交付物：与多个 learning rates 相关的 validation reward curves。

交付物：一个在 MATH 上达到至少 25% validation accuracy 的模型。

交付物：2 句简短讨论，说明你在其他 logged metrics 上注意到的趋势。

对于后续实验，你可以使用上述 sweep 中表现最好的 learning rate。

Baselines 的影响。使用上述超参数（除了换成你调好的 learning rate）继续，我们将研究 baselining 的影响。在 on-policy 设置下，我们比较以下 loss types：

- `no_baseline`
- `reinforce_with_baseline`

注意，默认超参数中 `use_std_normalization` 为 True。

**问题** (grpo\_baselines)：Baselining 的影响（2 分）（2 H100 小时）

分别使用 `reinforce_with_baseline` 和 `no_baseline` 训练 policy。

交付物：与每种 loss type 相关的 validation reward curves。

交付物：2 句简短讨论，说明你在其他 logged metrics 上注意到的趋势。

对接下来几个实验，使用上述实验中最优的 loss type。

长度归一化。正如在实现 `masked_mean` 时所暗示的，沿 sequence length 平均 losses 并不是必要的，甚至不一定正确。如何对 loss 求和是一个重要超参数，它会导致对 policy actions 的 credit attribution 方式不同。

让我们通过 Lambert [2024] 中的一个例子来说明这一点。观察 GRPO train step，首先获得 per-token policy gradient losses（暂时忽略 clipping）：

```
advantages # (batch_size, 1)
per_token_probability_ratios # (batch_size, sequence_length)
per_token_loss = -advantages * per_token_probability_ratios
```

这里我们已经将 advantages broadcast 到 sequence length。我们比较两种聚合 per-token losses 的方法：

- 我们实现的 `masked_mean`，它对每个 sequence 中未被 mask 的 tokens 求平均。
- 对每个 sequence 中未被 mask 的 tokens 求和，并除以一个常数标量（我们的 `masked_normalize` 方法支持 `constant_normalizer != 1.0`）[Liu et al., 2025, Yu et al., 2025]。

考虑一个例子：batch size 为 2，第一个 response 有 4 个 tokens，第二个 response 有 7 个 tokens。然后，我们可以看到这些归一化方法如何影响梯度。

```
from your_utils import masked_mean, masked_normalize

ratio = torch.tensor([
    [1, 1, 1, 1, 1, 1, 1,],
    [1, 1, 1, 1, 1, 1, 1,],
], requires_grad=True)
advs = torch.tensor([
    [2, 2, 2, 2, 2, 2, 2,],
    [2, 2, 2, 2, 2, 2, 2,],
])
masks = torch.tensor([
    # generation 1: 4 tokens
    [1, 1, 1, 1, 0, 0, 0,],
    # generation 2: 7 tokens
    [1, 1, 1, 1, 1, 1, 1,],
])
# Normalize with each approach
max_gen_len = 7
masked_mean_result = masked_mean(ratio * advs, masks, dim=1)
masked_normalize_result = masked_normalize(
```

```

ratio * advs, masks, dim=1, constant_normalizer=max_gen_len)
print("masked_mean", masked_mean_result)
print("masked_normalize", masked_normalize_result)
# masked_mean tensor([2., 2.], grad_fn=<DivBackward0>)
# masked_normalize tensor([1.1429, 2.0000], grad_fn=<DivBackward0>)
masked_mean_result.mean().backward()
print("ratio.grad", ratio.grad)
# ratio.grad:
# tensor([[0.2500, 0.2500, 0.2500, 0.2500, 0.0000, 0.0000, 0.0000],
# [0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429]])
ratio.grad.zero_()
masked_normalize_result.mean().backward()
print("ratio.grad", ratio.grad)
# ratio.grad:
# tensor([[0.1429, 0.1429, 0.1429, 0.1429, 0.0000, 0.0000, 0.0000],
# [0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429, 0.1429]])

```

**问题** (think\_about\_length\_normalization)：思考长度归一化（1分）

交付物：比较这两种方法（暂时不运行实验）。每种方法的优缺点是什么？有没有某些具体设置或例子中某种方法看起来更好？

现在，让我们用实证方式比较 `masked_mean` 与 `masked_normalize`。

**问题** (grpо\_length\_normalization)：长度归一化的影响（2分）（2 H100 小时）

交付物：通过端到端 GRPO training run 比较使用 `masked_mean` 和 `masked_normalize` 的归一化。报告 validation answer reward curves。评论发现，包括任何有明显趋势的其他 metrics。

提示：考虑与稳定性相关的 metrics，例如 gradient norm。

在后续实验中固定为表现更好的 length normalization 方法。

使用 group standard deviation 归一化。回忆一下我们对 `compute_group_normalized_rewards` 的标准实现（基于 Shao et al. [2024]、DeepSeek-AI et al. [2025]），其中按 group standard deviation 进行归一化。Liu et al. [2025] 指出，除以 group standard deviation 可能会给训练过程引入不想要的偏差：组内标准差较低的问题（例如太容易或太难，rewards 几乎全为 1 或全为 0）会在训练中获得更高权重。

Liu et al. [2025] 提出移除按 standard deviation 的归一化；我们已经在 `compute_group_normalized_rewards` 中实现这一选项，现在将测试它。

**问题** (grpо\_group\_standard\_deviation)：Standard deviation 归一化的影响（2分）（2 H100 小时）

交付物：比较 `use_std_normalization == True` 和 `use_std_normalization == False` 的性能。报告 validation answer reward curves。评论发现，包括任何有明显趋势的其他 metrics。

提示：考虑与稳定性相关的 metrics，例如 gradient norm。

在后续实验中固定为表现更好的 group normalization 方法。

Off-policy 与 on-policy。到目前为止，我们实验的超参数都是 on-policy：每个 rollout batch 只执行一个 gradient step，因此我们几乎就是在使用对 policy gradient 的“有原则”近似 `g_hat`（除了上面提到的 length 和 advantage normalization 选择）。

虽然这种方法在理论上合理且稳定，但效率不高。Rollouts 需要从 policy 进行缓慢 generation，因此是 GRPO 的主导成本；每个 rollout batch 只执行单个 gradient step 似乎很浪费，可能不足以显著改变 policy 行为。

现在我们将实验 off-policy training，即在每个 rollout batch 上执行多个 gradient steps（甚至多个 epochs）。

**问题** (grpо\_off\_policy)：实现 off-policy GRPO

交付物：实现 off-policy GRPO training。

根据你对完整 GRPO train loop

的实现，你可能已经具备了完成它的基础设施。如果没有，你需要实现以下内容：

- 能够对每个 rollout batch 执行多个 epochs 的 gradient steps，其中 epochs 数和每个 rollout batch 的 optimizer updates 由 `rollout_batch_size`、`epochs_per_rollout_batch` 和 `train_batch_size` 控制。
- 编辑主训练循环，在每次 rollout batch generation phase 之后、gradient steps 的 inner loop 之前，从 policy 获取 response logprobs——这些就是 `old_log_probs`。建议使用 `torch.inference_mode()`。
- 使用 "GRPO-Clip" loss type。

现在我们可以用 epochs 数和每个 rollout batch 的 optimizer updates 数来控制 off-policy 程度。

**问题** (grpo\_off\_policy\_sweep)：Off-policy GRPO 超参数 sweep (4 分) (12 H100 小时)

交付物：固定 `rollout_batch_size = 256`，选择一个 `epochs_per_rollout_batch` 与 `train_batch_size` 的范围做 sweep。首先在有限 GRPO steps (<50) 上做一个 broad sweep，以了解 performance landscape；然后在更多 GRPO steps (200) 上做一个更聚焦的 sweep。提供一个简短 experiment log，说明你选择这些范围的原因。

与 on-policy run (`epochs_per_rollout_batch = 1` 且 `train_batch_size = 256`) 进行比较，分别报告随 validation steps 变化以及随 wall-clock time 变化的 plots。

报告 validation answer reward curves。评论发现，包括任何有明显趋势的其他 metrics，例如 entropy 和 response length。将训练过程中模型 response 的 entropy 与 EI 实验中的观察进行比较。

提示：你需要改变 `gradient_accumulation_steps` 来保持 memory usage 恒定。

在 off-policy 设置中消融 clipping。回忆一下，GRPO-Clip 中 clipping 的目的是在单个 rollout batch 上执行多个 gradient steps 时，防止 policy 离旧 policy 太远。接下来，我们将在 off-policy 设置中消融 clipping，测试它到底有多必要。换言之，我们将使用 per-token loss：

$$- [\pi_{\theta}(\mathbf{o}_t | \mathbf{q}, \mathbf{o}_{<t}) / \pi_{\theta_{\text{old}}}(\mathbf{o}_t | \mathbf{q}, \mathbf{o}_{<t})] A_t. \quad (34)$$

**问题** (grpo\_off\_policy\_clip\_ablation)：Off-policy GRPO-Clip ablation (2 分) (2 H100 小时)

交付物：实现 unclipped per-token loss，作为新的 loss type "GRPO-No-Clip"。使用上一题中表现最好的 off-policy hyperparameters，运行 unclipped 版本的 loss。报告 validation answer reward curves。与 GRPO-Clip run 比较并评论发现，包括任何有明显趋势的其他 metrics，例如 entropy、response length 和 gradient norm。

提示词的影响。作为最后一个 ablation，我们将研究一个令人惊讶的现象：RL 使用的 prompt 会显著影响模型性能，具体取决于模型如何预训练。

我们不再使用 `cs336_alignment/prompts/r1_zero.prompt` 中的 R1-Zero prompt，而改用一个极其简单的 prompt，位于 `cs336_alignment/prompts/question_only.prompt`：

```
{question}
```

训练和验证时都使用该 prompt，并将 reward function（训练和验证都使用）改为位于 `cs336_alignment/drgrpo_grader.py` 的 `question_only_reward_fn`。

**问题** (grpo\_prompt\_ablation)：Prompt ablation (2 分) (2 H100 小时)

交付物：报告 R1-Zero prompt 和 question-only prompt 的 validation answer reward curves。各项 metrics 如何比较，包括任何有明显趋势的其他 metrics，例如 entropy、response length 和 gradient norm？尝试解释你的发现。

## 9 排行榜：MATH 上的 GRPO

作为必做部分的最后一部分，你将实验不同方法，目标是在 2 张 H100 GPU 上 4 小时训练时间内获得尽可能高的 validation rewards。

模型。我们继续使用 Qwen 2.5 Math 1.5B Base 模型。

数据集。我们继续使用集群上 `/data/a5-alignment/MATH/train.jsonl` 和 `/data/a5-alignment/MATH/validation.jsonl` 中的 MATH train 和 validation 数据集。你不能使用任何其他数据，也不能在来自更强模型的 reasoning chains 上做 SFT 等。你必须使用上面给定的 sampling hyperparameters (temperature 1.0、max tokens 1024) 在整个 validation set (所有 5K examples) 上报告 validation accuracy。你可以按自己想要的方式过滤 train set，或者设计数据 curriculum。验证时必须使用 R1-Zero prompt，并且验证时必须使用 starter code 中提供的 `r1_zero_reward_fn` reward function (如果愿意，你可以在训练期间开发另一个 reward function)。

算法。你可以自由调节超参数或完全改变训练算法，只要不使用外部数据或另一个模型 (如果你愿意，可以使用更多份该模型)。

系统优化。你可能会观察到，在我们上面的简单 GRPO 实现中，至少有一张 GPU 总是空闲。通过改善 pipeline 的系统特性，你很可能获得显著提升。例如，可以考虑 rollouts 或 training 使用更低精度、`torch.compile` 以及其他系统优化。你绝对不受限于将 vLLM 放在单个 device 上、将 train policy 放在另一个 device 上；我们鼓励你思考更好的并行化方式。

思路。有关可能改进的思路，可参考以下 repos：

- [veRL](#)
- [trl](#)
- [torchtune](#)
- [oat](#)

关于 KL divergence。我们还注意到，在上述实验中，我们没有加入相对于某个 reference model (通常是 frozen SFT 或 pretrained checkpoint) 的 KL divergence 项。在我们的实验和文献中的其他结果 [Liu et al., 2025, NTT123, 2025] 中，我们发现省略 KL 项对性能没有影响，同时还能节省 GPU 内存 (无需存储 reference model)。然而，许多 GRPO repos 默认包含它；只要你使用 Qwen 2.5 Math 1.5B Base 或通过你的算法获得的某个模型作为 reference，就鼓励你实验 KL 或其他形式的 regularization。

**问题** (leaderboard)：排行榜 (16 分) (16 H100 小时)

交付物：报告在 2 张 H100 GPU 上 4 小时训练内获得的 validation accuracy，以及 validation accuracy 随 wall-clock time 变化的截图，其中 x-axis 结束于  $\leq 4$  小时。提醒一下，我们对你的评估施加以下限制：

1. 你的 validation accuracy 应当是在整个 MATH validation set (所有 5K examples) 上的平均准确率。
2. 验证时必须使用 R1-Zero prompt。
3. 使用 vLLM 评估时必须使用 temperature 1.0 和 max tokens 1024。
4. 必须通过对 starter code 中提供的 `r1_zero_reward_fn` reward function 产生的 answer rewards 求平均来计算 validation accuracy。

## 10 结语

恭喜你完成本课程最后一次作业！你应该为自己的努力感到自豪。

我们希望你喜欢了通过从头构建现代语言模型主要组件来学习其基础原理的过程。

## 参考文献

以下参考文献保留原文条目，便于按论文标题、作者和 URL 精确检索。

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.12948>.

Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, Charles Sutton, and Augustus Odena. Show your work: Scratchpads for intermediate computation with language models, 2021. URL <https://arxiv.org/abs/2112.00114>.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL <https://arxiv.org/abs/2201.11903>.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. Star: Bootstrapping reasoning with reasoning, 2022. URL <https://arxiv.org/abs/2203.14465>.

Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search, 2017. URL <https://arxiv.org/abs/1705.08439>.

OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, Alex Iftimie, Alex Karpenko, Alexander Neitz, Alexander Prokofiev,

Alexander Wei, Allison Tam, Ally Bennett, Ananya Kumar, Andre Saraiva, Andrea Vallone, Andrew Duberstein, Andrew Kondrich, Andrey Mishchenko, Andy Applebaum, Angela Jiang, Ashvin Nair, Barret Zoph, Behrooz Ghorbani, Ben Rossen, Benjamin Sokolowsky, Boaz Barak, Bob McGrew, Borys Minaiev, Botao Hao, Bowen Baker, Brandon Houghton, Brandon McKinzie, Brydon Eastman, Camillo Lugaresi, Cary Bassin, Cary Hudson, Chak Ming Li, Charles de Bourcy, Chelsea Voss, Chen Shen, Chong Zhang, Chris Koch, Chris Orsinger, Christopher Hesse, Claudia Fischer, Clive Chan, Dan Roberts, Daniel Kappler, Daniel Levy, Daniel Selsam, David Dohan, David Farhi, David Mely, David Robinson, Dimitris Tsipras, Doug Li, Dragos Oprica, Eben Freeman, Eddie Zhang, Edmund Wong, Elizabeth Proehl, Enoch Cheung, Eric Mitchell, Eric Wallace, Erik Ritter, Evan Mays, Fan Wang, Felipe Petroski Such, Filippo Raso, Florencia Leoni, Foivos Tsimpourlas, Francis Song, Fred von Lohmann, Freddie Sulit, Geoff Salmon, Giambattista Parascandolo, Gildas Chabot, Grace Zhao, Greg Brockman, Guillaume Leclerc, Hadi Salman, Haiming Bao, Hao Sheng, Hart Andrin, Hessam Bagherinezhad, Hongyu Ren, Hunter Lightman, Hyung Won Chung, Ian Kivlichan, Ian O'Connell, Ian Osband, Ignasi Clavera Gilaberte, Ilge Akkaya, Ilya Kostikov, Ilya Sutskever, Irina Kofman, Jakub Pachocki, James Lennon, Jason Wei, Jean Harb, Jerry Twore, Jiacheng Feng, Jiahui Yu, Jiayi Weng, Jie Tang, Jieqi Yu, Joaquin Quiñero Candela, Joe Palermo, Joel Parish, Johannes Heidecke, John Hallman, John Rizzo, Jonathan Gordon, Jonathan Uesato, Jonathan Ward, Joost Huizinga, Julie Wang, Kai Chen, Kai Xiao, Karan Singhal, Karina Nguyen, Karl Cobbe, Katy Shi, Kayla Wood, Kendra Rimbach, Keren Gu-Lemberg, Kevin Liu, Kevin Lu, Kevin Stone, Kevin Yu, Lama Ahmad, Lauren Yang, Leo Liu, Leon Maksin, Leyton Ho, Liam Fedus, Lilian Weng, Linden Li, Lindsay McCallum, Lindsey Held, Lorenz Kuhn, Lukas Kondraciuk, Lukasz Kaiser, Luke Metz, Madelaine Boyd, Maja Trebacz, Manas Joglekar, Mark Chen, Marko Tintor, Mason Meyer, Matt Jones, Matt Kaufer, Max Schwarzer, Meghan Shah, Mehmet Yatbaz, Melody Y. Guan, Mengyuan Xu, Mengyuan Yan, Mia Glaese, Mianna Chen, Michael Lampe, Michael Malek, Michele Wang, Michelle Fradin, Mike McClay, Mikhail Pavlov, Miles Wang, Mingxuan Wang, Mira Murati, Mo Bavarian, Mostafa Rohaninejad, Nat McAleese, Neil Chowdhury, Neil Chowdhury, Nick Ryder, Nikolas Tezak, Noam Brown, Ofir Nachum, Oleg Boiko, Oleg Murk, Olivia Watkins, Patrick Chao, Paul Ashbourne, Pavel Izmailov, Peter Zhokhov, Rachel Dias, Rahul Arora, Randall Lin, Rapha Gontijo Lopes, Raz Gaon, Reah Miyara, Reimar Leike, Renny Hwang, Rhythm Garg, Robin Brown, Roshan James, Rui Shu, Ryan Cheu, Ryan Greene, Saachi Jain, Sam Altman, Sam Toizer, Sam Toyer, Samuel Miserendino, Sandhini Agarwal, Santiago Hernandez, Sasha Baker, Scott McKinney, Scottie Yan, Shengjia Zhao, Shengli Hu, Shibani Santurkar, Shraman Ray Chaudhuri, Shuyuan Zhang, Siyuan Fu, Spencer Papay, Steph Lin, Suchir Balaji, Suvansh Sanjeev, Szymon Sidor, Tal Broda, Aidan Clark, Tao Wang, Taylor Gordon, Ted Sanders, Tejal Patwardhan, Thibault Sottiaux, Thomas Degry, Thomas Dimson, Tianhao Zheng, Timur Garipov, Tom Stasi, Trapit Bansal, Trevor Creech, Troy Peterson, Tyna Eloundou, Valerie Qi, Vineet Kosaraju, Vinnie Monaco, Vitchyr Pong, Vlad Fomenko, Weiyi Zheng, Wenda Zhou, Wes McCabe, Wojciech Zaremba, Yann Dubois, Yinghai Lu, Yining Chen, Young Cha, Yu Bai, Yuchen He, Yuchen Zhang, Yunyun Wang, Zheng Shao, and Zhuohan Li. OpenAI o1 system card, 2024. URL <https://arxiv.org/abs/2412.16720>.

Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, Chuning Tang, Congcong Wang, Dehao Zhang, Enming Yuan, Enzhe Lu, Fengxiang Tang, Flood Sung, Guangda Wei, Guokun Lai, Haiqing Guo, Han Zhu, Hao Ding, Hao Hu, Hao Yang, Hao Zhang, Haotian Yao, Haotian Zhao, Haoyu Lu, Haoze Li, Haozhen Yu, Hongcheng Gao, Huabin Zheng, Huan Yuan, Jia Chen, Jianhang Guo, Jianlin Su, Jianzhou Wang, Jie Zhao, Jin Zhang, Jingyuan Liu, Junjie Yan, Junyan Wu, Lidong Shi, Ling Ye, Longhui Yu, Mengnan Dong, Neo Zhang, Ningchen Ma, Qiwei Pan, Qucheng Gong, Shaowei Liu, Shengling Ma, Shupeng Wei, Sihan Cao, Siying Huang, Tao Jiang, Weihao Gao, Weimin Xiong, Weiran He, Weixiao Huang, Wenhao Wu, Wenyang He, Xianghui Wei, Xianqing Jia, Xingzhe Wu, Xinran Xu, Xinxing Zu, Xinyu Zhou, Xuehai Pan, Y. Charles, Yang Li, Yangyang Hu, Yangyang Liu, Yanru Chen, Yejie Wang, Yibo Liu, Yidao Qin, Yifeng Liu, Ying Yang, Yiping Bao, Yulun Du, Yuxin Wu, Yuzhi Wang, Zaida Zhou, Zhaoji Wang, Zhaowei Li, Zhen Zhu, Zheng Zhang, Zhexu Wang, Zhilin Yang, Zhiqi Huang, Zihao Huang, Ziyao Xu, and Zonghan Yang. Kimi k1.5: Scaling reinforcement learning with llms, 2025. URL <https://arxiv.org/abs/2501.12599>.

Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL [https://proceedings.neurips.cc/paper\\_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf).

Hugging Face. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL <https://github.com/huggingface/open-r1>.

Weihaio Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025. URL <https://arxiv.org/abs/2503.18892>.

Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. Tinyzero. <https://github.com/Jiayi-Pan/TinyZero>, 2025. Accessed: 2025-01-24.

An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024. URL <https://arxiv.org/abs/2409.12122>.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021a. URL <https://arxiv.org/abs/2110.14168>.

Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2025. URL <https://arxiv.org/abs/2411.15124>.

Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding r1-zero-like training: A critical perspective, 2025. URL <https://arxiv.org/abs/2503.20783>.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. arXiv:2309.06180.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021b. arXiv:2110.14168.

David Dohan, Winnie Xu, Aitor Lewkowycz, Jacob Austin, David Bieber, Raphael Gontijo Lopes, Yuhuai Wu, Henryk Michalewski, Rif A. Saurous, Jascha Sohl-dickstein, Kevin Murphy, and Charles Sutton. Language model cascades, 2022. URL <https://arxiv.org/abs/2207.10342>.

Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, Wolfgang Macherey, Arnaud Doucet, Orhan Firat, and Nando de Freitas. Reinforced self-training (rest) for language modeling, 2023. URL <https://arxiv.org/abs/2308.08998>.

Joshua Achiam. Spinning up in deep reinforcement learning. 2018a.

Nathan Lambert. Reinforcement learning from human feedback, 2024. URL <https://rlhfrbook.com>.

Sheldon M Ross. Simulation. academic press, 2022.

Thomas Degris, Martha White, and Richard S. Sutton. Off-policy actor-critic, 2013. URL <https://arxiv.org/abs/1205.4839>.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.

Joshua Achiam. Simplified ppo-clip objective, 2018b. URL <https://drive.google.com/file/d/1PDzn9RPvaXjFZkGeapMHbHGiWWW20Ey/view>.

Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiase Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.

NTT123. Grpo-zero. <https://github.com/policy-gradient/GRPO-Zero>, 2025. Accessed: 2025-05-22.